

Diagnóstico e Setup de API .NET Core local

Porta localhost • Swagger • ERR_CONNECTION_CLOSED • Certificado HTTPS dev

Visual Studio 2022 | ASP.NET Core + Kestrel | Angular | Windows

Este guia consolida o aprendizado de uma sessão real de troubleshooting: como descobrir a porta em que a API sobe, por que o Swagger pode não aparecer, como diagnosticar o erro **net::ERR_CONNECTION_CLOSED** entre Angular e API, e o passo a passo para gerar e confiar no certificado HTTPS de desenvolvimento via Certificate Manager.

Sumário

1. Descobrindo a porta localhost da API
2. Configurando o Swagger no Program.cs
3. O Swagger não aparece — o caso do ASPNETCORE_ENVIRONMENT
4. Erro net::ERR_CONNECTION_CLOSED (status 0) no Angular
5. Certificado HTTPS de desenvolvimento (passo a passo)
6. Importando o certificado no Certificate Manager (certmgr)
7. CORS — o próximo suspeito depois da conexão
8. Checklist final de troubleshooting
9. Referência rápida de comandos

1. Descobrindo a porta localhost da API

No .NET, a porta usada ao rodar a API vem de alguns lugares. Vale conhecer a ordem de precedência para saber onde olhar — e qual valor realmente vale.

1.1. launchSettings.json (ponto de partida)

Fica em **Properties/launchSettings.json**. É o arquivo que o Visual Studio usa ao rodar com F5/Ctrl+F5. Cada *profile* pode ter portas diferentes; a porta efetiva depende do profile selecionado no dropdown ao lado do botão play verde.

```
{
  "profiles": {
    "BNPP.Hercules.Api": {
      "commandName": "Project",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7025;http://localhost:5184",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "applicationUrl": "http://localhost:36099",
      "sslPort": 44360
    }
  }
}
```

1.2. A fonte da verdade: o log ao subir

Independentemente da configuração, quando a API sobe ela imprime exatamente onde está escutando. Essa é a confirmação definitiva:

```
INF Now listening on: https://localhost:7025
INF Now listening on: http://localhost:5184
INF Application started. Press Ctrl+C to shut down.
INF Hosting environment: Debug
```

Como o profile influencia

Se o profile selecionado é **commandName: "Project"** (Kestrel), valem as portas do **applicationUrl** (7025 / 5184). As portas do IIS Express (36099 / 44360) só entram em jogo se você rodar com o profile **IIS Express**.

1.3. Sobrescritas em código

Vale conferir o **Program.cs** por chamadas que têm precedência sobre o launchSettings:

`builder.WebHost.UseUrls(...)`, `app.Run("http://localhost:XXXX")` ou a variável de ambiente `ASPNETCORE_URLS`.

2. Configurando o Swagger no Program.cs

Atenção à versão do .NET

A partir do **.NET 9** a Microsoft removeu o Swashbuckle (a UI do Swagger) do template padrão. O template novo só gera o documento OpenAPI em JSON, sem interface visual. Confira o `<TargetFramework>` no `.csproj` antes de decidir o caminho.

2.1. .NET 6 / 7 / 8 — Swashbuckle

Os dois pontos essenciais são registrar (`AddSwaggerGen`) e habilitar o middleware (`UseSwagger + UseSwaggerUI`). A UI fica em `/swagger`.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();           // 1. registro

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();                       // 2. gera /swagger/v1/swagger.json
    app.UseSwaggerUI();                     // serve a UI em /swagger
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

2.2. .NET 9+ — OpenAPI nativo (sem UI)

O pacote `Microsoft.AspNetCore.OpenApi` gera apenas o JSON em `/openapi/v1.json`. Para ter UI, você pode continuar usando o Swashbuckle (funciona normalmente no .NET 9) ou apontar uma UI moderna como o **Scalar** para o documento gerado.

```
builder.Services.AddOpenApi();           // nativo .NET 9+
...
app.MapOpenApi();                         // expoe /openapi/v1.json
app.MapScalarApiReference();              // UI alternativa (pacote Scalar.AspNetCore) em /scalar/v1
```

3. O Swagger não aparece — o caso do ASPNETCORE_ENVIRONMENT

Este foi o problema concreto da sessão. Tudo configurado, mas a tela do Swagger não subia. A causa estava no ambiente.

Sintoma

O log mostra `Hosting environment: Debug` e o Swagger nunca aparece em `/swagger`, mesmo com o código correto.

3.1. Causa raiz

No `launchSettings`, a linha do ambiente `Development` estava comentada e o ambiente fixado num nome customizado (`"Debug"`):

```
"environmentVariables": {  
  //"ASPNETCORE_ENVIRONMENT": "Development"  
  "ASPNETCORE_ENVIRONMENT": "Debug"  
}
```

O método `app.Environment.IsDevelopment()` compara a string com **"Development"** exatamente. Como o ambiente é **"Debug"**, o método retorna **false** e o bloco que habilita o Swagger nunca executa.

3.2. As três soluções

Abordagem	O que fazer
Opção 1 — voltar a Development	Descomente a linha original no <code>launchSettings</code> : <code>"ASPNETCORE_ENVIRONMENT": "Development"</code>
Opção 2 — manter "Debug"	Inclua o ambiente customizado na checagem do <code>Program.cs</code> : <code>if (app.Environment.IsDevelopment() app.Environment.IsEnvironment("Debug"))</code>
Opção 3 — habilitar sempre	Tire o <code>if</code> e chame <code>UseSwagger()</code> / <code>UseSwaggerUI()</code> direto. Comum em APIs internas.

Abrir o /swagger automaticamente

Para o navegador abrir direto na tela do Swagger ao rodar, adicione `"launchUrl": "swagger"` no profile que você usa (o profile **Project** da sessão não tinha essa linha — só o IIS Express tinha).

4. Erro net::ERR_CONNECTION_CLOSED (status 0) no Angular

O Angular (em `http://localhost:4200`) chama a API em `https://localhost:7025/api/authentication/authenticate` e recebe um `HttpErrorResponse` com status 0.

Como ler o erro

status: 0 e **statusText: "Unknown Error"** significam que a requisição não chegou na API — morreu no nível de conexão, antes do seu código C#. O **ERR_CONNECTION_CLOSED** indica que a conexão foi aberta e fechada abruptamente (tipicamente no handshake TLS).

4.1. Causas prováveis, em ordem

#	Causa	Diagnóstico
1	API não está rodando	O debug parou ou a API caiu. Confirme o <code>Now listening on: https://localhost:7025</code> no log agora .
2	Certificado HTTPS dev não confiável	Suspeito nº 1 para ERR_CONNECTION_CLOSED em HTTPS local. O handshake TLS falha e a conexão fecha. (Ver seção 5.)
3	Mismatch de porta http vs https	Se o front aponta <code>http://...:7025</code> numa porta que só fala https, a conexão fecha. Confirme o <code>apiUrl</code> no <code>environment.ts</code> .
4	CORS	Menos provável aqui: CORS mostraria mensagem específica e a API teria recebido a requisição. Status 0 + conexão fechada aponta mais para TLS. (Ver seção 7.)

O teste decisivo (faça isto primeiro)

Abra direto no navegador: `https://localhost:7025/swagger`.

- **Erro de conexão/certificado também?** → problema é HTTPS/certificado da API → vá para a seção 5.
- **Swagger abriu normal?** → API saudável → o problema está na ponte Angular→API (URL, proxy, navegador).

REFUSED vs CLOSED

ERR_CONNECTION_REFUSED = ninguém escutando na porta. **ERR_CONNECTION_CLOSED** = algo respondeu e fechou — o que aponta mais para TLS/certificado do que para 'API desligada'.

5. Certificado HTTPS de desenvolvimento (passo a passo)

5.1. Caminho automático (tente primeiro)

No PowerShell (não precisa ser admin):

```
# Remove certificados dev antigos/corrompidos
dotnet dev-certs https --clean

# Gera um novo e tenta registrar como confiável
dotnet dev-certs https --trust
```

Se aparecer o popup do Windows pedindo para instalar o certificado, clique **Sim**. Funcionou? Pule para a validação (5.5). Se o popup for bloqueado por política corporativa ou não aparecer, faça o caminho manual.

5.2. "Falha ao inserir certificado" no Debug do VS (limpeza via MMC)

Sintoma

Ao dar **Debug no Visual Studio**, o VS tenta criar/confiar no certificado HTTPS e exibe "**Falha ao inserir certificado**" (ou "There was an error trusting the certificate"). O `dotnet dev-certs https --clean` não resolve sozinho.

Causa: já existe um certificado `localhost` corrompido, duplicado ou travado no store **Pessoal** (Current User\Personal). A nova tentativa de inserção colide com o resíduo antigo e falha. A correção é remover o certificado manualmente pelo MMC e deixar o VS (ou o `dotnet dev-certs`) regenerar do zero.

Passo	Ação
1	Abra o MMC como administrador : menu Iniciar → digite mmc → botão direito → Executar como administrador .
2	Arquivo → Adicionar/Remover Snap-in → Certificados → Adicionar → Minha conta de usuário → Concluir → OK .
3	Navegue até Console Root\Certificates - Current User\Personal\Certificates.
4	Localize o certificado <code>localhost</code> → botão direito → Excluir .
5	Cheque também ... \Trusted Root Certification Authorities\Certificates e remova entradas <code>localhost</code> antigas (evita certificado órfão/duplicado).
6	Rode <code>dotnet dev-certs https --clean</code> e depois <code>--trust</code> , ou apenas dê Debug de novo no VS para ele regenerar.

Por que como administrador

O snap-in de **"Minha conta de usuário"** em si não exige admin, mas abrir o MMC elevado evita erros de permissão e permite, na mesma janela, adicionar o snap-in de **"Conta de computador"** caso precise inspecionar o store da máquina (LocalMachine).

5.3. Exportar o certificado manualmente

O dev cert nasce no store **Pessoal** (CurrentUser\My), mas para ser confiável precisa também estar nas **Autoridades de Certificação Raiz Confiáveis** (Trusted Root). Exportamos a parte pública (.cer) para importar lá.

```
# Garante que o certificado dev existe
dotnet dev-certs https

# Exporta a parte publica para um .cer
dotnet dev-certs https --export-path "$env:USERPROFILE\Desktop\localhost-dev.cer" --format PEM
```

5.4. Alternativa: gerar do zero com PowerShell

Caso o dotnet dev-certs não seja viável no seu ambiente:

```
$cert = New-SelfSignedCertificate -DnsName "localhost" `
    -CertStoreLocation "cert:\CurrentUser\My" `
    -FriendlyName "localhost dev cert" `
    -NotAfter (Get-Date).AddYears(2) `
    -KeyExportPolicy Exportable `
    -KeyUsage DigitalSignature,KeyEncipherment `
    -TextExtension @"(2.5.29.37={text}1.3.6.1.5.5.7.3.1)"

Export-Certificate -Cert $cert -FilePath "$env:USERPROFILE\Desktop\localhost-dev.cer"
```

5.5. Validar

```
dotnet dev-certs https --check --trust
```

Saída esperada: algo como *"A valid certificate was found and is trusted"*. Depois: **(1)** reinicie a API, **(2)** feche o navegador completamente e reabra (não só a aba — o navegador cacheia decisões de TLS), **(3)** acesse `https://localhost:7025/swagger`.

6. Importando o certificado no Certificate Manager (certmgr)

Store do usuário, não da máquina

O certificado precisa ir para o store do **usuário atual** (CurrentUser), não da máquina. Por isso usamos `certmgr.msc` (usuário) e não `certlm.msc` (máquina). O Kestrel procura o dev cert no store pessoal do usuário.

6.1. Via interface gráfica (certmgr.msc)

Passo	Ação
1	Win + R → digite <code>certmgr.msc</code> → Enter (abre o gerenciador do usuário atual).
2	Expanda Autoridades de Certificação Raiz Confiáveis → Certificados .
3	Botão direito em Certificados → Todas as Tarefas → Importar...
4	Avançar → Procurar → selecione <code>localhost-dev.cer</code> . (Mude o filtro para "Todos os arquivos" se não aparecer.)
5	Avançar → confirme repositório Autoridades de Certificação Raiz Confiáveis → Avançar → Concluir .
6	Se surgir aviso de segurança sobre instalar certificado raiz → Sim .

6.2. Via PowerShell (equivalente, mais rápido)

```
Import-Certificate `
  -FilePath "$env:USERPROFILE\Desktop\localhost-dev.cer" `
  -CertStoreLocation "Cert:\CurrentUser\Root"
```

`Cert:\CurrentUser\Root` é exatamente o "Autoridades de Certificação Raiz Confiáveis" do usuário atual.

Ambiente corporativo (GPO)

Políticas de grupo podem reverter ou bloquear a instalação no Trusted Root. Se o certificado sumir do store ou continuar não confiável após tudo, é sinal de que uma política está removendo — nesse caso é necessário acionar o time de infraestrutura/segurança. É restrição administrativa, não técnica.

7. CORS — o próximo suspeito depois da conexão

Quando a conexão TLS passar a funcionar, o próximo ponto a checar é o CORS — porque o Angular (origem `localhost:4200`) e a API (`localhost:7025`) são origens diferentes. Diferente do `ERR_CONNECTION_CLOSED`, um erro de CORS aparece com mensagem explícita no console e a API recebe a requisição.

```
// Program.cs
builder.Services.AddCors(o => o.AddPolicy("Front", p => p
    .WithOrigins("http://localhost:4200")
    .AllowAnyHeader()
    .AllowAnyMethod()));

var app = builder.Build();

// Ordem importa: UseCors antes de UseAuthorization/MapControllers
app.UseCors("Front");
app.UseAuthorization();
app.MapControllers();
```

Ordem do middleware

`UseCors` precisa vir **antes** de `UseAuthorization` e do mapeamento dos endpoints, senão a política não é aplicada.

8. Checklist final de troubleshooting

#	Verificação	Ação
1	API está rodando?	Olhe <code>Now listening on: https://localhost:7025</code> no log.
2	Swagger abre direto no navegador?	<code>https://localhost:7025/swagger</code> . Separa problema de API vs Angular.
3	Swagger não sobe?	Cheque o ambiente (seção 3): <code>IsDevelopment()</code> vs <code>ASPNETCORE_ENVIRONMENT="Debug"</code> .
4	Erro de conexão/cert no navegador?	<code>dotnet dev-certs https --clean / --trust</code> + importar no <code>certmgr</code> (seções 5–6).
5	VS falha ao inserir certificado no Debug?	Limpe o <code>localhost</code> em <code>Current User\Personal</code> via MMC admin e regenere (seção 5.2).
6	Reiniciou tudo?	Reinicie a API E feche/reabra o navegador completamente (cache de TLS).
7	Front aponta para https na porta certa?	Confirme <code>apiUrl = https://localhost:7025</code> no <code>environment.ts</code> .
8	Conexão OK mas erro novo?	Agora sim verifique CORS (seção 7) e o status HTTP real (401/404/500).

9. Referência rápida de comandos

Tarefa	Comando
Verificar dev cert	<code>dotnet dev-certs https --check --trust</code>
Limpar dev certs	<code>dotnet dev-certs https --clean</code>
Gerar + confiar	<code>dotnet dev-certs https --trust</code>
Exportar .cer (PEM)	<code>dotnet dev-certs https --export-path "...\\localhost-dev.cer" --format PEM</code>
Abrir certmgr (usuário)	<code>certmgr.msc</code>
Importar no Trusted Root	<code>Import-Certificate -FilePath "...\\localhost-dev.cer" -CertStoreLocation "Cert:\\CurrentUser\\Root"</code>
Adicionar Swashbuckle	<code>dotnet add package Swashbuckle.AspNetCore</code>
Adicionar Scalar (.NET 9+)	<code>dotnet add package Scalar.AspNetCore</code>

Resumo do raciocínio central: o **status 0 / ERR_CONNECTION_CLOSED** quase nunca é bug de código — é a requisição não chegando na API. O teste de abrir **/swagger** direto no navegador é o que separa, em um único passo, "problema de API/certificado" de "problema de Angular". E o Swagger sumido geralmente é o ambiente (**IsDevelopment()**) e não o Swagger em si.